

Tema 12. Límites teóricos de computación.

12.1. Problema de parada

Definición 12.1.1. Problema asociado al predicado.

Problem	Predicate	Function
solvable (<i>resoluble</i>)	decidable	$\in TREC$
partially solvable (<i>parcialmente resoluble</i>)	enumerable	$\in REC$
unsolvable (<i>irresoluble</i>)	undecidable	$\notin TREC$
totally unsolvable (<i>totalmente irresoluble</i>)	not enumerable	$\notin REC$

Definición 12.1.2. Predicado H

Dado $Q \in \text{WHILE}$, $g = \text{while2N}(Q)$, y $\underline{x} \in \mathbb{N}^*$, $H(g, \underline{x}) \Leftrightarrow F_Q(\underline{x}) \in \mathbb{N}$

Nota. $H(g, \underline{x})$ es verdad si y solo si Q se para con la entrada $\underline{x}$.

Definición 12.1.3. Predicado H^n

Dada $Q \in \text{WHILE} : F_Q \in F(\text{WHILE})^n$, $g = \text{while2N}(Q)$, y $\underline{x} \in \mathbb{N}^n$.
 $H^n(g, \underline{x}) \Leftrightarrow F_Q(\underline{x}) \in \mathbb{N}$.

Nota. A partir de ahora, g será referido como 'un programa' (es decir, el programa que codifica), o 'el número' de un programa.

Teorema 12.1.1. El problema H^1 no es solucionable

Probado Por la contradicción:

$$H^1 \text{ solvable} \Rightarrow H^1 \text{ decidable} \Rightarrow \exists h^1 \in TREC : h^1 = X_{V_{H^1}}$$

$$h^1(g, x) = \begin{cases} 0 & \text{if } \overline{H^1(g, x)} \\ 1 & \text{if } H^1(g, x) \end{cases}$$

Vamos a considerar la funcion

$$\text{bomb}(x) = \begin{cases} 1 & \text{if } h^1(x, x) = 0 \\ \uparrow & \text{if } h^1(x, x) = 1 \end{cases}$$

$Bomb = (1, s)$

$s :$

```

while  $H_1(X_1, X_1) \neq 0$  do
   $X_1 := X_1$ 
od
 $X_1 := X_2 + 1;$ 

```

If $b = while_{2N}(Bomb)$ then

$h^1(b, b) = 1 \Rightarrow bomb(b) = \uparrow \Rightarrow h^1(b, b) = 0 \Rightarrow \Leftarrow$

$h^1(b, b) = 0 \Rightarrow bomb(b) = 1 \Rightarrow h^1(b, b) = 1 \Rightarrow \Leftarrow$

Colorario 12.1.1. H no es solucionable.

Nota. H^1 no es solucionable $\Rightarrow H^n$ no es solucionable $\Rightarrow H$ no es solucionable

Proposición 12.1.1. H^1 es parcialmente solucionable.

Prueba. Vamos a considerar

$f : \mathbb{N}^2 \rightarrow \mathbb{N}$
 $f = \sigma(U[REC^1])$

$$P_f = H^1 \xRightarrow[\substack{\uparrow \\ f \in REC}]{\substack{\uparrow \\ f \in REC}} H^1 \in \text{PRED}(REC) \Rightarrow H^1 \text{ is partially solvable.}$$

Nota. Siendo H^1 numerable significa que todos los pares

$(g, x) \in \mathbb{N} : F_{N2while(g)}(x) \in \mathbb{N}$ puede ser generados. Puede ser hecho simulando cada programa con una entrada dada para un numero de pasos, y comprobando si se ha parado.

Colorario 12.1.2. H es parcialmente solucionable.

Nota. H^1 parcialmente solucionable $\Rightarrow H^n$ parcialmente solucionable $\Rightarrow H$ parcialmente solucionable.

12.2. La funcion del castor afanoso

Definición 12.2.1. Longitud de un programa While.

$length : \text{WHILE} \rightarrow \mathbb{N}$
 $length((n, s)) = |s|_{do} + |s|_{:=}$

Definición 12.2.2. WHILE^n

$\text{WHILE}^n = \{Q \in \text{WHILE} : length(Q) = n\}$

Nota. WHILE^n es el conjunto de programas While de longitud n.

Definición 12.2.3. Funcion de castor afanoso (Σ)

$$\Sigma : \mathbb{N} \rightarrow \mathbb{N}$$

$$\Sigma(n) = \max\{F_Q(0) : F_Q \in F(\text{WHILE})^1 \wedge Q \in \text{WHILE}^n \wedge F_Q(0) \in \mathbb{N}\}$$

Proposición 12.2.1. $\Sigma(n+1) > \Sigma(n)$

Prueba. Vamos a considerar $Q = (1, s) \in \text{WHILE}^n \wedge F_Q(0) = \Sigma(n)$, entonces

$$Q' = (1, s')$$

s' :

s ;

$X1 := X1 + 1$

$$\text{length}(Q') = n + 1 \wedge F_{Q'}(0) = \Sigma(n) + 1 \Rightarrow$$

$$\Sigma(n+1) \geq \Sigma(n) + 1 > \Sigma(n) \Rightarrow \Sigma(n+1) > \Sigma(n)$$

Proposición 12.2.2. $\exists Q \in \text{WHILE}^k : f = F_Q \Rightarrow \Sigma(n+k) \geq f(n)$

Prueba. Sea considerado $Q = (1, s) \in \text{WHILE}^k \wedge F_Q = f$, entonces

$$Q' = (1, s')$$

s' :

$$\left. \begin{array}{l} X1 := X1 + 1; \\ \dots \\ X1 := X1 + 1; \end{array} \right\} n \text{ times}$$

$$Q' \in \text{WHILE}^{n+k} \wedge F_{Q'}(0) = f(n) \Rightarrow \Sigma(n+k) \geq f(n)$$

Teorema 12.2.1. $\Sigma \notin F(\text{WHILE})$

Prueba, Por contradicción: vamos a considerar.

$$\left. \begin{array}{l} \Sigma \in F(\text{WHILE}) \\ \text{let } g(n) = 2n \xRightarrow[\text{example 9.1.2}]{\uparrow} g \in F(\text{WHILE}) \\ \text{let } f(n) = \Sigma(2n) \end{array} \right\} \Rightarrow f \in F(\text{WHILE}) \Rightarrow$$

$$\exists Q \in \text{WHILE}^k : F_Q = f \xRightarrow[\text{proposition 12.2.2}]{\uparrow} \Sigma(n+k) \geq f(n), \forall n \in \mathbb{N} \xRightarrow[\text{proposition 12.2.1}]{\uparrow} \Sigma(n+k) \geq \Sigma(2n)$$

$$\Sigma(n+k) \geq \Sigma(2n), \forall n \in \mathbb{N} \xRightarrow[\text{proposition 12.2.1}]{\uparrow} \Sigma(2k+1) \geq \Sigma(2k+2) \xRightarrow[\text{proposition 12.2.1}]{\uparrow} \Sigma(2k+1) \geq \Sigma(2k+2)$$

Proposición 12.2.3. $\Sigma \notin F(\text{WHILE}) \Rightarrow H^1$ no es solucionable

Prueba. Por contradicción: vamos a considerar $\Sigma \notin F(\text{WHILE}) \wedge H^1$ es solucionable.

Dado que hay un numero finito de programas (equivalentes) de una longitud dada, entonces se puede encontrar el programa que se detiene (con entrada cero) para cada longitud, y por lo tanto la mayor salida.

Esto implica $\Sigma \in F(\text{WHILE}) \Rightarrow \Leftarrow$